



Laboratório 3 - Implementando o decoder

No Laboratório 2, nós dissecamos o Encoder e entendemos como o Transformer extrai o contexto de uma frase inteira simultaneamente. Agora precisamos construir um sistema que gere texto de forma fluente, uma palavra por vez, garantindo que o modelo não cometa o erro de "olhar para o futuro" durante seu processamento. Neste laboratório, vocês construirão os blocos matemáticos centrais do Decoder.

Objetivos de Aprendizagem:

1. Dominar a álgebra linear por trás do mascaramento causal (Look-Ahead Masking).
2. Implementar a ponte de integração estrutural entre duas redes neurais (Atenção Cross-Attention).
3. Simular um loop de geração auto-regressiva.

Tarefa 1: Implementando a Máscara Causal (Look-Ahead Mask)

Fundamento: No treinamento paralelizado em GPUs, a frase de destino completa entra no Decoder. Para impedir que a palavra na posição i atenda à posição $i+1$, injetamos uma máscara matricial M antes do cálculo do Softmax.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V$$

O que deve ser codificado:

1. Crie uma função chamada `create_causal_mask(seq_len)` que receba o tamanho de uma sequência (ex: `seq_len = 5`).
2. A função deve retornar uma matriz quadrada $[\text{seq_len}, \text{seq_len}]$ onde a parte triangular inferior (e a diagonal principal) contém zeros (0), e a parte triangular superior contém infinito negativo (-infinito).
3. Prova Real: Multiplique matrizes fictícias Q e K , adicione a sua máscara M ao resultado, e aplique a função Softmax. Imprima o resultado no console para provar que as probabilidades das palavras futuras se tornaram estritamente 0.0.

Tarefa 2: A Ponte Encoder-Decoder (Cross-Attention)





Fundamento: Diferente do Self-Attention, onde as representações derivam do mesmo texto, o Encoder-Decoder Attention cruza fronteiras. O estado atual da geração interage com a "memória" lida pelo Encoder.

O que deve ser codificado:

1. Crie tensores fictícios para simular a saída do Encoder: `encoder_output` com dimensão `[batch_size=1, seq_len_frances=10, d_model=512]`.
2. Crie tensores fictícios simulando o que o Decoder já gerou: `decoder_state` com dimensão `[batch_size=1, seq_len_ingles=4, d_model=512]`.
3. Escreva a função `cross_attention(encoder_out, decoder_state)`.
 - Projete `decoder_state` usando uma matriz de pesos arbitrária para virar a Query (Q).
 - Projete `encoder_out` usando matrizes arbitrárias para virar as Keys (K) e Values (V).
4. Calcule a equação do Scaled Dot-Product Attention. Note que, nesta etapa em específico, não usamos máscara causal, pois o modelo deve ter permissão para olhar para a frase original do Encoder por completo.

Tarefa 3: Simulando o Loop de Inferência Auto-Regressivo

Fundamento: O modelo de linguagem trabalha como um laço de repetição condicionado na produção anterior. Após as sub-camadas, o vetor final de 512 dimensões sofre uma projeção linear para o tamanho do vocabulário, seguida de um Softmax, emitindo uma distribuição de probabilidades para o próximo token.

O que deve ser codificado:

1. Crie uma função genérica de Mock `generate_next_token(current_sequence, encoder_out)` que receba a lista de tokens gerados até agora (ex: `["<START>", "O", "rato"]`) e simule a passagem pelo Decoder.
2. A função deve cuspir um vetor de probabilidades do tamanho de um vocabulário fictício (ex: `V = 10.000`).
3. O Loop: Escreva um laço `while` (o loop de inferência) que chame esta função iterativamente.
 - A cada passo, aplique `argmax` para selecionar a palavra com a maior probabilidade.
 - Adicione (`append`) a nova palavra na lista de contexto.
 - O loop deve ser interrompido imediatamente e o código deve imprimir a frase final se o modelo gerar o token fictício `<EOS>`.

